

BÁO CÁO SAU BUỔI THỰC HÀNH

Môn học: Java Web Service

Tên dự án: Employee Management API – RESTful Web Service

Thời gian thực hành: 120p

Họ và tên sinh viên: Nguyễn Công Gia Huy

Mã sinh viên: PTIT-HN-163

Lớp: HN-K24-CNTT2

I. NỘI DUNG ĐÃ THỰC HÀNH

- Phân tích yêu cầu nghiệp vụ và tài liệu đặc tả của hệ thống Employee Management API.
- Thiết kế và xây dựng RESTful API quản lý nhân viên sử dụng Spring Boot.
- Cấu hình thực thể Employee và thao tác dữ liệu với H2 In-Memory Database thông qua Spring Data JPA.
- Định nghĩa cấu trúc Response Body tùy biến (bao gồm các trường status, message, data, timestamp) để chuẩn hóa dữ liệu trả về trên Postman.
- Tích hợp lập trình hướng khía cạnh (Spring AOP) để tự động hóa Logging hệ thống với 3 loại Advice: @Before, @AfterReturning, @Around.
- Triển khai hệ thống ghi log phân cấp (INFO, WARN, ERROR) sử dụng SLF4J.
- Viết Unit Test cho tầng Controller (@WebMvcTest) và tầng Service (@ExtendWith) bằng Mockito để đảm bảo chất lượng code.

II. CÔNG VIỆC ĐÃ LÀM

1. Công việc cá nhân (Tập trung phát triển tầng Controller & Định dạng API Response)

- **Xây dựng EmployeeController:** Triển khai toàn bộ các endpoint RESTful để tiếp nhận các request từ client và gọi xuống tầng Service xử lý nghiệp vụ:
 - GET /api/employees - Lấy danh sách toàn bộ nhân viên.
 - GET /api/employees/{id} - Lấy chi tiết nhân viên theo ID sử dụng @PathVariable.
 - POST /api/employees - Thêm nhân viên mới, tiếp nhận và validate dữ liệu đầu vào bằng @Valid và @RequestBody.
 - PUT /api/employees/{id} - Cập nhật toàn bộ thông tin nhân viên theo ID.
 - DELETE /api/employees/{id} - Xóa nhân viên khỏi hệ thống.
- **Chuẩn hóa cấu trúc dữ liệu phản hồi (Response Body):** Thiết kế class wrapper dạng ApiResponse<T> chung để đảm bảo mọi kết quả trả về Postman đều hiển thị tường minh gồm:
 - status: Mã trạng thái HTTP (200, 201, 400, 404,...).
 - message: Thông báo cụ thể (Ví dụ: "Fetch employees successfully", "Employee not found with id: X").
 - data: Dữ liệu JSON trả về (Object hoặc Array).
 - timestamp: Thời gian phản hồi request.
- **Xử lý Exception tập trung:** Kết hợp @RestControllerAdvice và @ExceptionHandler để bắt các lỗi RuntimeException (khi không tìm thấy ID) hoặc MethodArgumentNotValidException (khi dữ liệu validate lỗi) để map sang HTTP Status phù hợp (404 Not Found, 400 Bad Request).
- **Ghi log tại Controller:** Áp dụng SLF4J để ghi log mức INFO ngay khi nhận request gửi tới (Ví dụ: log.info("GET /api/employees called")).
- **Viết Unit Test cho Controller:** Thực hiện 4 test case sử dụng MockMvc kết hợp @MockBean để giả lập dữ liệu từ EmployeeService nhằm kiểm thử trạng thái HTTP Status và cấu trúc JSON trả về.

2. Công việc nhóm

- Tham gia thảo luận, phân tích yêu cầu nghiệp vụ hệ thống từ tài liệu của khách hàng.
- Thống nhất thiết kế cấu trúc project theo mô hình kiến trúc đa tầng chuẩn: controller, service, repository, entity, dto, config, aspect, exception.
- Phối hợp xây dựng tầng cấu trúc lõi: Thực thể Employee (gồm các trường: id, fullName, department, salary) và giao diện EmployeeRepository kế thừa JpaRepository.
- Phối hợp cùng các thành viên hiện thực hóa logic nghiệp vụ ở tầng Service (xử lý thêm, sửa, xóa, kiểm tra logic nghiệp vụ ném ra biệt lệ).
- Hỗ trợ cấu hình Spring AOP để thực hiện đo hiệu năng thời gian chạy của Controller (@Around), log tên phương thức (@Before), và log kết quả trả về của Service (@AfterReturning).
- Thực hiện kiểm thử toàn diện (Integration Test) các API trên Postman với cả trường hợp dữ liệu hợp lệ và dữ liệu lỗi.

III. KẾT QUẢ ĐẠT ĐƯỢC

- Hiểu rõ và áp dụng thuần thục nguyên tắc thiết kế RESTful API (sử dụng đúng các phương thức GET, POST, PUT, DELETE và HTTP Status Code).
- Nắm chắc tư duy tách biệt mã nguồn theo kiến trúc 3 tầng (Controller - Service - Repository), giúp mã nguồn dễ bảo trì và mở rộng.
- Thành thạo cách tùy biến cấu trúc dữ liệu Response dạng JSON giúp Client/Frontend dễ bóc tách dữ liệu.
- Biết cách quản lý cơ sở dữ liệu gọn nhẹ bằng H2 In-Memory Database kết hợp cùng Spring Data JPA để rút ngắn thời gian phát triển hệ thống.
- Hiểu sâu sắc cơ chế hoạt động của Spring AOP và cách ứng dụng ghi log thực tế bằng thư viện SLF4J phục vụ quá trình giám sát (monitoring).
- Nâng cao tư duy kiểm thử phần mềm thông qua việc viết Unit Test bằng Mockito & MockMvc cho cả tầng Service lẫn Controller.
- Link Git:
 - https://github.com/GiaHuy2306/JavaWebService_Session13_MiniProject

IV. KHÓ KHĂN VÀ VẤN ĐỀ GẶP PHẢI

- Gặp khó khăn ban đầu khi cấu hình AOP @Around để đo đạc thời gian thực thi (chưa xử lý tốt đối tượng ProceedingJoinPoint).
- Quá trình validate dữ liệu bằng @NotBlank và @Min đôi khi trả về cấu trúc lỗi mặc định của Spring khá rườm rà, mất thời gian để format lại qua @RestControllerAdvice về chuẩn response mong muốn.
- Đôi lúc còn nhầm lẫn trong việc thiết lập dữ liệu mẫu tự động chèn vào H2 Database khi khởi chạy ứng dụng (cấu hình qua file data.sql).

V. KINH NGHIỆM RÚT RA

- Việc thiết kế một Response Body đồng nhất (chứa status, message, data) từ đầu giúp quá trình kiểm thử bằng Postman cũng như tích hợp Frontend trở nên vô cùng rõ ràng, giảm thiểu hiểu lầm.
- Luôn tuân thủ nguyên tắc: Controller chỉ giữ vai trò điều hướng, tiếp nhận validate request và gọi Service; không bao giờ viết logic nghiệp vụ trực tiếp tại Controller.
- Tận dụng tối đa AOP Logging giúp mã nguồn sạch sẽ hơn, loại bỏ việc lặp đi lặp lại mã lệnh System.out.println() hay log.info() ở quá nhiều hàm.
- Việc viết Unit Test giúp phát hiện sớm các lỗi logic (như trả sai HTTP Status code hoặc thiếu trường dữ liệu) ngay trước khi tiến hành deploy.

